

performance_logger.py - What Was Built

AI-Based Personal Grooming Assistant - FYP report sections 6.8 & 6.9

What I built

performance_logger.py (new, project root)

Tracks FPS, CNN inference time, analysis-to-lock time, and RAM; prints a live summary every 5 seconds; saves to performance_results.txt on demand; watches app.log's file size to flag new errors; and writes stress_test_log.txt with the 7 stress-test cases pre-listed.

face_shape_model.py (small addition, no logic changed)

Added a stopwatch around just the CNN's invoke()/predict() call (not the cropping step), exposed via get_last_inference_ms(). Face shape classification behavior is unchanged.

main.py (small hook-ins)

Replaced the old ad-hoc FPS/memory print block (which printed every 60 frames/60s) with one call to perf_logger.tick(...) each frame, reusing the FPS number main.py already computed. Added the 'L' key to save results.

How to run it

- Run python main.py as usual - the performance logger starts automatically alongside the app (you'll see "[perf] Created stress_test_log.txt" in the console).
- Watch the terminal - every 5 seconds you'll see a line like: "FPS: 22 | Inference: 45ms | Memory: 410MB"
- Press L at any point to append a timestamped snapshot of the current averages (FPS, inference ms, analysis-to-lock ms, memory, error count) to performance_results.txt. Do this a few times during normal use to get numbers for Section 6.8.
- If app.log grows (a real per-frame error got logged), you'll see "[perf] app.log grew -- error #N logged, check app.log for the traceback" in the terminal - use that to tell if a crash happened during a stress test.

Stress testing (Section 6.9)

Open stress_test_log.txt (created at project root, or regenerate anytime with python performance_logger.py) - it has all 7 cases pre-listed with blank Result (PASS/FAIL) and Notes lines to fill in by hand while testing:

- No face in view -> should not crash, should show a prompt
- Sideways face -> frame should be skipped
- Very dark or very bright light -> skin tone should show low confidence or stay empty
- Two faces in frame -> should use the largest face only
- Rapid key presses (pressing N, P, D quickly) -> should not crash
- Long run, about 30 minutes -> memory and speed should stay stable
- Model files removed/renamed -> app should fall back to rule-based mode with a visible badge

Skipped (by design)

An on-screen counter for app.log errors - used terminal + file logging instead, since a real UI element would mean touching ui.py. Can be added later if it needs to be visible in-frame too.